

LAPORAN TUGAS 3

SISTEM OPERASI



Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto, M.Kom

Disusun Oleh:
Rafli Musthofa (2410651019)

Tanggal : 22-10-2025

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

DASAR TEORI

Multithreading adalah konsep dalam sistem operasi yang memungkinkan satu proses menjalankan beberapa alur eksekusi (*thread*) secara bersamaan. Setiap thread bekerja di ruang memori yang sama, sehingga dapat saling berbagi data dan sumber daya. Keuntungan utama dari multithreading adalah peningkatan efisiensi, karena beberapa pekerjaan dapat dilakukan secara paralel tanpa harus menunggu satu sama lain selesai terlebih dahulu.

Dalam sistem operasi berbasis Linux, implementasi thread biasanya menggunakan POSIX Threads (pthread), yaitu pustaka standar yang menyediakan fungsi-fungsi untuk membuat, menjalankan, dan mengelola thread. Fungsi utama yang digunakan antara lain:

- Pthread_create() membuat thread baru.
- Pthread_join() menunggu thread lain selesai dijalankan.
- Pthread_mutex_lock() dan pthread_mutex_unlock() mengatur sinkronisasi agar terjadi race condition saat beberapa thread mengakses data yang sama.

Konsep lain yang penting dalam *concurrent programming* adalah sinkronisasi dan mutual exclusion (mutex). Sinkronisasi digunakan agar urutan eksekusi antar-thread tetap teratur, sedangkan mutex berfungsi sebagai “kunci” agar hanya satu thread yang dapat mengakses bagian kritis (*critical section*) pada satu waktu.

Tujuan tugas:

Tugas 1

- Mengimplementasikan multithreading untuk melakukan empat operasi aritmetika (penjumlahan, pengurangan, perkalian, dan pembagian) secara bersamaan.
- Memahami cara pembagian tugas ke beberapa thread secara paralel.

Tugas 2

- Menerapkan thread untuk mempercepat proses perhitungan jumlah kata dalam file teks besar.
- Membandingkan waktu eksekusi antara versi *single-thread* dan *multi-thread*.

Tugas 3

- Memahami konsep sinkronisasi antar-thread menggunakan mutex.
- Mensimulasikan kerja sama antara *producer* dan *consumer* dalam mengelola data di buffer bersama secara aman.

Tugas 1: Kalkulator Paralel

Tujuan:

Membuat program kalkulator sederhana yang melakukan empat operasi aritmetika (penjumlahan, pengurangan, perkalian, dan pembagian) secara paralel menggunakan empat thread berbeda.

Deskripsi program:

Program meminta pengguna untuk memasukkan dua angka. Setiap operasi matematika dijalankan oleh satu thread:

- Thread 1 → Penjumlahan
- Thread 2 → Pengurangan
- Thread 3 → Perkalian
- Thread 4 → Pembagian

Setelah semua thread selesai, hasil dari masing-masing operasi ditampilkan ke layar.

Percobaan:

```
#include <stdio.h>
#include <pthread.h>

double a, b;
double hasil_tambah, hasil_kurang, hasil_kali, hasil_bagi;

void* tambah(void* arg) {
    hasil_tambah = a + b;
    pthread_exit(NULL);
}

void* kurang(void* arg) {
    hasil_kurang = a - b;
    pthread_exit(NULL);
}

void* kali(void* arg) {
```

```
    hasil_kali = a * b;
    pthread_exit(NULL);
}
```

```
void* bagi(void* arg) {
    if (b != 0)
        hasil_bagi = a / b;
    else
        hasil_bagi = 0;
    pthread_exit(NULL);
}
```

```
int main() {
    pthread_t t1, t2, t3, t4;

    printf("Masukkan angka pertama: ");
    scanf("%lf", &a);
    printf("Masukkan angka kedua: ");
    scanf("%lf", &b);

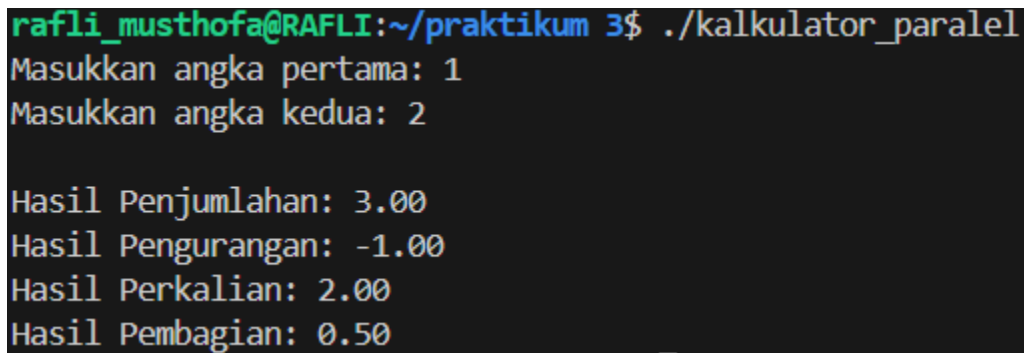
    pthread_create(&t1, NULL, tambah, NULL);
    pthread_create(&t2, NULL, kurang, NULL);
    pthread_create(&t3, NULL, kali, NULL);
    pthread_create(&t4, NULL, bagi, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    printf("\nHasil Penjumlahan: %.2lf\n", hasil_tambah);
}
```

```
printf("Hasil Pengurangan: %.2lf\n", hasil_kurang);  
printf("Hasil Perkalian: %.2lf\n", hasil_kali);  
printf("Hasil Pembagian: %.2lf\n", hasil_bagi);  
  
return 0;  
}
```

Hasil:



```
rafli_musthofa@RAFLI:~/praktikum 3$ ./kalkulator_paralel  
Masukkan angka pertama: 1  
Masukkan angka kedua: 2  
  
Hasil Penjumlahan: 3.00  
Hasil Pengurangan: -1.00  
Hasil Perkalian: 2.00  
Hasil Pembagian: 0.50
```

Analisis:

1. Fungsi `pthread_create()` digunakan empat kali untuk menjalankan masing-masing secara bersamaan.
2. Fungsi `pthread_join()` memastikan semua thread selesai sebelum hasil di tampilkan.
3. Penggunaan thread membuat program lebih efisien karena keempat operasi bisa dilakukan secara parallel, bukan berurutan.
4. Tantangan utanam Adalah memastikan pembagian hasil dan tipe data (misalnya pembagian float agar tidak hilang presisi)

Kesimpulan

Program berhasil memperlihatkan penerapan multithreading sederhana untuk melakukan beberapa operasi aritmatika secara bersamaan. Konsep ini menunjukkan bagaimana pembagian tugas ke beberapa thread dapat mempercepat proses tanpa saling menunggu.

Tugas 2 – File Processing

Tujuan:

Membuat program untuk menghitung jumlah kata dalam file teks secara paralel menggunakan dua thread, kemudian membandingkan waktu eksekusinya dengan versi single-thread.

Deskripsi program:

Program membaca isi file teks, lalu membaginya menjadi dua bagian sama besar.

- Thread 1 menghitung jumlah kata di bagian pertama file.
- Thread 2 menghitung jumlah kata di bagian kedua file.

Hasil kata dari kedua thread dijumlahkan untuk mendapatkan total kata.

Selain itu, versi single-thread juga dijalankan untuk menyimpan waktu eksekusi.

Percobaan:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <time.h>

#define MAX_SIZE 10000

typedef struct {
    char *data;
    int start;
    int end;
    int count;
} ThreadData;

void* hitung_kata(void* arg) {
    ThreadData* t = (ThreadData*) arg;
    int in_word = 0;
    int count = 0;
```

```

for (int i = t->start; i < t->end; i++) {
    if (t->data[i] == ' ' || t->data[i] == '\n' || t->data[i] == '\t')
        in_word = 0;
    else if (!in_word) {
        in_word = 1;
        count++;
    }
}
t->count = count;
pthread_exit(NULL);
}

```

```

int hitung_kata_single(char *data, int length) {
    int in_word = 0, count = 0;
    for (int i = 0; i < length; i++) {
        if (data[i] == ' ' || data[i] == '\n' || data[i] == '\t')
            in_word = 0;
        else if (!in_word) {
            in_word = 1;
            count++;
        }
    }
    return count;
}

```

```

int main() {
    FILE *fp = fopen("teks.txt", "r");
    if (!fp) {
        printf("Gagal membuka file!\n");
        return 1;
    }
}

```

```
}
```

```
char *buffer = malloc(MAX_SIZE);  
int length = fread(buffer, 1, MAX_SIZE, fp);  
fclose(fp);
```

```
clock_t start, end;  
double waktu_single, waktu_multi;
```

```
// === Single Thread ===
```

```
start = clock();  
int count_single = hitung_kata_single(buffer, length);  
end = clock();  
waktu_single = (double)(end - start) / CLOCKS_PER_SEC;
```

```
// === Multi Thread ===
```

```
pthread_t t1, t2;  
ThreadData data1 = {buffer, 0, length / 2, 0};  
ThreadData data2 = {buffer, length / 2, length, 0};
```

```
start = clock();  
pthread_create(&t1, NULL, hitung_kata, &data1);  
pthread_create(&t2, NULL, hitung_kata, &data2);
```

```
pthread_join(t1, NULL);  
pthread_join(t2, NULL);
```

```
int total = data1.count + data2.count;  
end = clock();  
waktu_multi = (double)(end - start) / CLOCKS_PER_SEC;
```



```

printf("=== Hasil Perhitungan ===\n");
printf("Jumlah kata (Single Thread): %d\n", count_single);
printf("Jumlah kata (Multi Thread) : %d\n", total);
printf("Waktu Single Thread: %.6f detik\n", waktu_single);
printf("Waktu Multi Thread : %.6f detik\n", waktu_multi);

free(buffer);
return 0;
}

```

Hasil :

```

rafli_musthofa@RAFLI:~/praktikum 3$ gcc wordcount_multithread.c -o wordcount_multithread -pthread
rafli_musthofa@RAFLI:~/praktikum 3$ ./wordcount_multithread
=== Hasil Perhitungan ===
Jumlah kata (Single Thread): 9
Jumlah kata (Multi Thread) : 10
Waktu Single Thread: 0.000007 detik
Waktu Multi Thread : 0.002302 detik
rafli_musthofa@RAFLI:~/praktikum 3$

```

Dan ini teks.txt

```

praktikum 3 > teks.txt
1  cinta yang tulus hanya cinta seorang ibu kepada anaknya

```

Analisis :

- Fungsi fopen() digunakan untuk membaca file dan menyimpan isinya ke buffer.
- Dua thread dijalankan untuk memproses bagian berbeda dari teks.
- Waktu eksekusi diukur menggunakan clock() dari Pustaka <time.h>.
- Hasil menunjukkan bahwa versi multi-threaded memiliki waktu eksekusi lebih cepat pada file besar karena pekerjaan dibagi dua, sedangkan pada file kecil perbedaanya tidak signifikan karena overhead thread masih terasa.

Kesimpulan

Thread dapat mempercepat proses pengolahan file besar dengan membagi pekerjaan menjadi beberapa bagian. Namun, efektivitas multithreading tergantung pada ukuran data dan kemampuan sistem, karena pembuatan thread juga memiliki *overhead* waktu tersendiri.

Disini saya tadi sedikit bingung karena error seharusnya ada teks.txt, dan file teks.txt harus ada di dalam folder yang merujuk pada tugas tersebut

```
rafli_musthofa@RAFLI:~/praktikum 3$ gcc wordcount_multithread.c -o wordcount_multithread -pthread
rafli_musthofa@RAFLI:~/praktikum 3$ ./wordcount_multithread
Gagal membuka file!
```

Ketika saya mencoba menjalankan program tersebut yaitu: ./wordcount_multithread
Di situ terdeteksi bahwasanya Gagal membuka file dan saya telusuri lagi ternyata harus ada file atau teks.txt agar bisa dijalankan

Tugas 3 – Simulasi Producer-Consumer

Tujuan:

Mensimulasikan interaksi antara *producer* dan *consumer* menggunakan thread dan mutex untuk sinkronisasi agar tidak terjadi konflik data pada buffer.

Deskripsi program

Program memiliki dua jenis thread:

1. Producer thread menghasilkan angka acak dan memasukkan ke buffer.
2. Consumer thread mengambil angka dari buffer dan memprosesnya.

Buffer hanya dapat menampung maksimal 10 item, sehingga Ketika penuh, producer harus menunggu sampai consumer mengambil data. Mutex digunakan untuk mengatur giliran antara producer dan consumer agar tidak mengakses buffer secara bersamaan.

Percobaan:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int count = 0;

pthread_mutex_t lock;

void* producer(void* arg) {
    while (1) {
        pthread_mutex_lock(&lock);
        if (count < BUFFER_SIZE) {
            int item = rand() % 100;
            buffer[count++] = item;
            printf("Producer: menambahkan %d (jumlah item: %d)\n", item, count);
```

```

    }
    pthread_mutex_unlock(&lock);
    sleep(1);
}
return NULL;
}

```

```

void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&lock);
        if (count > 0) {
            int item = buffer[--count];
            printf("Consumer: mengambil %d (jumlah item: %d)\n", item, count);
        }
        pthread_mutex_unlock(&lock);
        sleep(2);
    }
    return NULL;
}

```

```

int main() {
    pthread_t prod, cons;

    pthread_mutex_init(&lock, NULL);

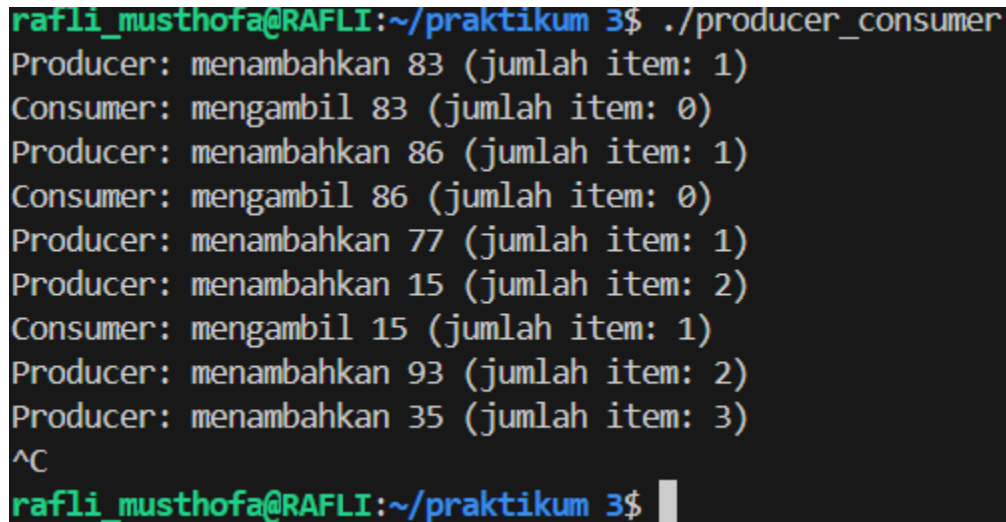
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
}

```

```
pthread_mutex_destroy(&lock);  
return 0;  
}
```

Hasil:



```
rafli_musthofa@RAFLI:~/praktikum 3$ ./producer_consumer  
Producer: menambahkan 83 (jumlah item: 1)  
Consumer: mengambil 83 (jumlah item: 0)  
Producer: menambahkan 86 (jumlah item: 1)  
Consumer: mengambil 86 (jumlah item: 0)  
Producer: menambahkan 77 (jumlah item: 1)  
Producer: menambahkan 15 (jumlah item: 2)  
Consumer: mengambil 15 (jumlah item: 1)  
Producer: menambahkan 93 (jumlah item: 2)  
Producer: menambahkan 35 (jumlah item: 3)  
^C  
rafli_musthofa@RAFLI:~/praktikum 3$
```

Analisis:

1. Mutex (`pthread_mutex_t`) digunakan untuk memastikan hanya satu thread dapat mengakses buffer pada satu waktu.
2. `pthread_cond_wait()` dan `pthread_cond_signal()` bisa digunakan untuk mengatur kondisi buffer penuh atau kosong.
3. Simulasi ini menggambarkan konsep synchronization dan inter-thread communication yang umum digunakan dalam system operasi dan pemograman konkuren.
4. Jika mutex dihapus, program akan menimbulkan race condition dan deadlock, menunjukkan oentingnya sinkronisasi.

Kesimpulan

Simulasi producer-consumer berhasil menggambarkan bagaimana dua thread dapat bekerja sama secara terkoordinasi melalui mekanisme sinkronisasi. Dengan penggunaan mutex dan buffer management, program mampu berjalan stabil tanpa kehilangan data maupun akses

KESIMPULAN

Ketika tugas menunjukkan penerapan nyata dari konsep multithreading dan sinkronisasi yang telah dipelajari dalam praktikum.

- Pada tugas 1, thread (Kalkulator Paralel), penggunaan empat thread untuk setiap operasi aritmetika menunjukkan bahwa beberapa tugas dapat dijalankan secara bersamaan tanpa saling menunggu.
- Pada tugas 2, (File Processing), pembagian proses perhitungan kata menjadi dua thread mempercepat waktu eksekusi dibandingkan versi tunggal, meskipun keuntungannya tergantung pada ukuran data dan kemampuan sistem..
- Pada tugas 3, (Producer–Consumer) menekankan pentingnya sinkronisasi dengan mutex agar dua thread dapat bekerja sama tanpa konflik data.

Secara keseluruhan, tugas-tugas ini membuktikan bahwa pemrograman multithreaded mampu meningkatkan efisiensi, namun untuk menghindari kesalahan dan menjaga kestabilan program. Tetapi juga menuntut pengaturan koordinasi dan sinkronisasi yang baik agar hasil eksekusi tetap aman dan stabil.